PW-DEV-016 · DOCUMENTO DE INGENIERIA

PitWall — Guía de desarrollo

De cero a un sistema funcionando: herramientas, módulos, pruebas, ejecución y depuración

VERSION DEL DOCUMENTO
1.3

ESTADO
Vigente

FECHA
2026-09-22

AUTOR
Luis Miguel

Documento generado automaticamente a partir de `16-DEVELOPMENT-GUIDE.md`. El fichero Markdown es la unica fuente de verdad; este PDF es un artefacto derivado y no debe editarse por separado.

PitWall es un proyecto no oficial sin afiliacion con Electronic Arts Inc., Codemasters, la FIA ni Formula One World Championship Limited. Las marcas citadas pertenecen a sus respectivos titulares y se emplean unicamente de forma descriptiva.

Indice

1. Herramientas

- 1.1. Obligatorias
- 1.2. Recomendadas
- 1.3. Lo que hace falta del juego

2. Instalación

- 2.1. Presets disponibles
- 2.2. Verificación de la instalación

3. Estructura del repositorio

4. Primer módulo: pw::core

- 4.1. Qué construir
- 4.2. La prueba que define el proyecto
- 4.3. Verificación

5. Segundo módulo: pw::proto — el más delicado

- 5.1. Procedimiento de transcripción
- 5.2. Orden recomendado
- 5.3. Verificación del bloque

6. Pruebas

- 6.1. Ejecución
- 6.2. Fuzzing
- 6.3. Cobertura
- 6.4. Trazabilidad

7. Ejecución

- 7.1. Modo directo
- 7.2. Modo replay
- 7.3. Frontend en desarrollo
- 7.4. Todo junto

8. Depuración

- 8.1. Núcleo
- 8.2. La técnica que resuelve casi todo
- 8.3. Frontend
- 8.4. Registro dirigido

9. Captura de sesiones UDP

- 9.1. Configuración del juego
- 9.2. Capturas necesarias
- 9.3. Después de capturar

10. Replay

11. Frontend

- 11.1. Órdenes
- 11.2. Tipos compartidos
- 11.3. Reglas de rendimiento

12. Integración

12.1. Flujo de trabajo
12.2. Lista de comprobación previa a integrar
12.3. Normas de Git
12.4. Documentación

13. Pruebas manuales

13.1. Sesión de conducción (cada hito)
13.3. Sesión con pw-listen y el juego real (F1.1-13)
13.4. Recepción desde la LAN (DoD de F1.1)
13.2. Configuración en consola (al implementar RF-003)
13.3. Tras un parche del juego

14. Resolución de problemas frecuentes

15. Por dónde empezar mañana

Guía de desarrollo

Esta guía se lee **de arriba abajo** la primera vez. Cada sección deja el proyecto en un estado verificable.

---

1. Herramientas

1.1. Obligatorias

Herramienta	Versión mínima	Para qué	Comprobación
Visual Studio 2022 con «Desarrollo para el escritorio con C++»	17.10	Compilador MSVC, depurador	`cl` en el terminal de desarrollo
WebView2 Runtime (Evergreen)	actual	Alojar la interfaz dentro de la ventana	Windows 11 lo trae de serie; comprobar en «Aplicaciones instaladas»
SDK de WebView2	fijado por manifiesto	API de C++ del control	Se resuelve al configurar CMake
CMake	3.28	Construcción	`cmake --version`
Git	2.40	Control de versiones	`git --version`
Node.js	22 LTS	Frontend y generación de PDF	`node --version`
Python	3.11	Utilidades del repositorio	`python --version`

1.2. Recomendadas

Herramienta	Para qué
LLVM / Clang para Windows	Segunda opinión del compilador, desinfectantes, fuzzing
Visual Studio Code con extensiones de C++ y CMake	Edición y depuración ligera
Wireshark	Ver los datagramas cuando nada más funciona
Herramientas de desarrollo del WebView2	Depurar la interfaz **dentro de la ventana** (sólo en compilación de desarrollo)
Un navegador basado en Chromium	Sólo para comparar rendimiento con una línea base. **No es una forma de uso soportada** (OUT-10)

1.3. Lo que hace falta del juego

Elemento	Cuándo
EA SPORTS F1 25 con el 2026 Season Pack	Sólo para **capturar** las sesiones de referencia (fase F1.4) y para la validación final
La especificación oficial de EA en `spec/`	Descargada del hilo oficial. **No se versiona**

> A partir de la fase F1.4 el juego deja de ser necesario para el desarrollo diario. Ése es el objetivo de todo el diseño de replay.

---

2. Instalación

```
git clone <url> pitwall
cd pitwall

# 1. Dependencias de C++ (manifiesto)
cmake --preset debug          # resuelve y compila las dependencias la primera vez

# 2. Compilar el núcleo
cmake --build --preset debug

# 3. Pruebas
ctest --preset debug --output-on-failure

# 4. Frontend
cd ui && npm ci && cd ..

# 5. Herramientas de documentación
cd tools && npm ci && cd ..
```

2.1. Presets disponibles

Preset	Uso
`debug`	Desarrollo diario, con aserciones
`release`	Medición de rendimiento y distribución
`asan`	Desinfectante de direcciones
`tsan`	Desinfectante de hilos (requiere Clang)
`fuzz`	Compilación de objetivos de fuzzing (requiere Clang)

2.2. Verificación de la instalación

Dos comprobaciones: que el núcleo escucha, y que la ventana abre.

```
# 1. El núcleo, sin interfaz
cmake --build --preset debug --target pw-listen
./build/debug/bin/pw-listen --port 20777            # juego en este mismo PC
./build/debug/bin/pw-listen --bind 0.0.0.0 --port 20777   # consola u otro PC de la LAN

# 2. La aplicación completa
cd ui && npm run build && cd ..
cmake --build --preset debug --target PitWall
./build/debug/bin/PitWall.exe
```

La primera debe imprimir `pw-listen — escuchando en 127.0.0.1:20777`, porque **el `bind` por defecto es `127.0.0.1`** (`net.bind`). Para recibir de otra máquina hay que pedir `--bind 0.0.0.0` explícitamente. Si el juego está emitiendo, aparece una línea por segundo con el estado, el ritmo y los contadores, y `Ctrl+C` la detiene con un resumen final.

La segunda debe abrir **una ventana propia** con la interfaz dentro, y estar ya escuchando sin que nadie pulse nada. Si se abriera un navegador, algo está mal configurado.

Si falla el bind: el mensaje indicará la causa. Las tres habituales son puerto ocupado por otra herramienta de telemetría, cortafuegos de Windows bloqueando, y falta de permisos en puertos por debajo de 1024 (no aplicable a 20777).

---

3. Estructura del repositorio

Detalle completo en `17-PROJECT-STRUCTURE.md`. Lo mínimo para orientarse:

```
pitwall/
├── core/          núcleo C++: src/ (módulos pw::*), apps/, include/, tests/
│   └── apps/      PitWall.exe (shell + núcleo) · pitwall-cli (núcleo sin interfaz)
├── ui/            frontend React + TypeScript
├── tools/         utilidades: md2pdf, generadores, scripts
├── docs/          los 18 documentos + PDF generados
├── spec/          especificación oficial de EA (NO versionado)
├── tests/         fixtures y escenarios compartidos
└── CMakePresets.json
```

Regla que no se negocia: `spec/` está en `.gitignore`. La especificación de EA se provee «for reference purposes only» y no se redistribuye (`01-REQUIREMENTS.md` §26.3).

---

4. Primer módulo: `pw::core`

Es el módulo donde se establece el estilo del proyecto. Merece hacerse despacio.

4.1. Qué construir

1. `Result<T,E>` y los tipos fuertes (`VehicleIndex`, `LapNumber`, `SessionKey`, `BranchId`).
2. `Channel<T>` con las cinco procedencias.

3. `monotonicNowNs()`.

4.2. La prueba que define el proyecto

```
// core/tests/test_channel.cpp
TEST_CASE("Channel exige comprobar known() antes de usar el valor", "[core]") {
    auto restricted = Channel<float>::restricted();
    REQUIRE_FALSE(restricted.known());
    REQUIRE(restricted.provenance() == Provenance::Restricted);
    REQUIRE(restricted.valueOr(-1.f) == -1.f);
}

// Prueba de NO compilación, verificada por el sistema de construcción
// static_assert(!std::is_convertible_v<Channel<float>, float>,
//               "Channel no debe convertirse implicitamente a su valor");
```

La prueba de no compilación es lo que convierte el principio P1 en una garantía del compilador (ADR-009). Merece la pena configurarla bien la primera vez.

4.3. Verificación

```
cmake --build --preset debug --target pw_core_tests
./build/debug/bin/pw_core_tests
```

---

5. Segundo módulo: `pw::proto` — el más delicado

Aquí se transcribe la especificación. Un error aquí produce datos falsos que parecen reales.

5.1. Procedimiento de transcripción

Para cada paquete, **en este orden**:

1. Abrir la especificación oficial en el struct correspondiente.
2. Transcribir campo a campo, **sin reordenar y sin «mejorar» nombres**.
3. Añadir el `static_assert` de tamaño con el valor oficial.
4. Compilar. **Si falla, el error está en la transcripción, no en la especificación.**
5. Añadir la comprobación aritmética `cabecera + N × porCoche + extras`.
6. Añadir `static_assert` de offset para 2-3 campos representativos.
7. Capturar el fixture real y escribir su `.json` de valores esperados **a mano**.
8. Escribir la prueba unitaria.

> **Consejo de un error real y frecuente.** El paso 4 tiene una trampa: cuando el `static_assert` falla, la tentación es cambiar el número esperado hasta que compile. Es exactamente lo contrario de lo que hay que hacer. El número oficial es la verdad; el struct es lo que está mal.

5.2. Orden recomendado

Bloque	Paquetes	Por qué en este orden
1	Header, 6 Car Telemetry, 7 Car Status	Los más usados; validan el mecanismo completo
2	2 Lap Data, 4 Participants	Necesarios para el cronometraje y la política de restricción
3	0 Motion, 13 Motion Ex	Introducen el desescalado de cuantizados
4	1 Session, 3 Event	Los más grandes y con más enumerados
5	16 Car Telemetry 2	Exclusivo de 2026
6	5, 8, 9, 10, 11, 12, 14, 15	El resto

Cerrar los `static_assert` de cada bloque antes de pasar al siguiente evita acumular 17 errores de transcripción a la vez.

5.3. Verificación del bloque

```
cmake --build --preset debug --target pw_proto_tests
```

Los `static_assert` fallan en compilación, así que **una compilación limpia ya es media prueba superada**.

---

6. Pruebas

6.1. Ejecución

```
ctest --preset debug --output-on-failure          # todo
ctest --preset debug -R "proto"                   # sólo protocolo
ctest --preset debug -R "RF-211"                  # por requisito
ctest --preset asan --output-on-failure           # con desinfectante de direcciones
ctest --preset tsan -R "concurrency"              # con desinfectante de hilos
```

6.2. Fuzzing

```
cmake --build --preset fuzz --target pw_decoder_fuzzer
./build/fuzz/bin/pw_decoder_fuzzer tests/corpus/ -max_total_time=60
```

Toda entrada que provoque un fallo se **añade permanentemente** al corpus como caso de regresión.

6.3. Cobertura

```
python tools/coverage.py --preset debug --min 80 \
    --modules pw_decoder pw_normalize pw_state
```

6.4. Trazabilidad

```
python tools/traceability.py
```

Falla si algún requisito `MUST` de `01-REQUIREMENTS.md` no tiene ninguna prueba etiquetada con su identificador (RNF-085).

---

7. Ejecución

7.1. Modo directo

```
./build/debug/bin/pitwall --config config/dev.toml
```

Con la configuración por defecto: abre su ventana, aloja la interfaz en el WebView2 y escucha UDP en `127.0.0.1:20777`. **No hay ninguna URL que abrir**: el transporte interno usa un puerto efímero que ni se configura ni se muestra.

7.2. Modo replay

```
./build/debug/bin/pitwall --replay tests/scenarios/2026_race_start.f1raw --speed 1x
```

Es el modo de trabajo habitual. No requiere el juego.

7.3. Frontend en desarrollo

```
cd ui && npm run dev                                  # Vite, con recarga en caliente
# En otra consola, con uiSource = "dev-server" en config/dev.toml:
./build/debug/bin/PitWall.exe --config config/dev.toml
```

El WebView2 apunta al servidor de Vite, así que hay **recarga en caliente dentro de la propia ventana de PitWall**. Es el modo de trabajo normal para la interfaz.

> **Abrir la URL de Vite en un navegador no funciona, y es correcto que no funcione.** La interfaz necesita el objeto `window.__pitwall` que inyecta el shell (endpoint efímero y token). Fuera de su anfitrión muestra un error explicándolo, en lugar de intentar adivinar una dirección.

7.4. Todo junto

```
python tools/dev.py --replay tests/scenarios/2026_race_start.f1raw --speed 0.5x
```

Arranca Vite y después `PitWall.exe` en modo replay apuntando al servidor de desarrollo. Se abre **la ventana de PitWall**, no un navegador.

---

8. Depuración

8.1. Núcleo

Síntoma	Primer sitio donde mirar
No llegan datagramas	*UDP Diagnostics* → si está a cero, el problema es de red, no del código
`bad_size` creciendo	El par (esperado, observado) en el log dice qué paquete y cuánto se desvía
`unknown_format` creciendo	Otro emisor en el mismo puerto, o formato nuevo de EA
Valores absurdos en un paquete	Casi siempre un campo mal transcrito: revisar los `static_assert` de offset
Vueltas duplicadas	Ramas de flashback. Reproducir el escenario con registro `TRACE`
Caída en el decoder	Reproducir con el desinfectante de direcciones; añadir la entrada al corpus
Latencia alta	Histogramas de `queue_wait_ns` por cola: señalan qué etapa se atasca

8.2. La técnica que resuelve casi todo

```
# 1. Reproducir el fallo con captura activada
./build/debug/bin/pitwall --record

# 2. Recortar la ventana de interés
pw-capture slice captura.f1raw --from 00:12:00 --to 00:13:00 --out fallo.f1raw

# 3. Reproducir a voluntad, en el depurador, con TRACE
./build/debug/bin/pitwall --replay fallo.f1raw --speed 0.25x --log-level trace
```

Un fallo intermitente se convierte en un fallo reproducible. **Es la razón de ser de ADR-007.**

8.3. Frontend

Síntoma	Dónde mirar
Sin datos en la interfaz	Herramientas de desarrollo del WebView2, pestaña de red: ¿está abierta la conexión? ¿llegan tramas?
Caídas de fotogramas	Grabación de rendimiento: buscar reconciliaciones de React durante el dibujo
Memoria creciendo	Instantáneas de memoria comparadas: buscar búferes circulares no liberados
«Se ha abierto fuera de su anfitrión»	Se está usando un navegador en vez de `PitWall.exe`
La ventana abre en blanco	Recursos mal mapeados, o falta ejecutar `npm run build`
No abre nada al hacer doble clic	Falta el WebView2 Runtime, o ya había una instancia y se activó su ventana
Valores que no cambian	Comprobar la antigüedad del dato: puede ser el backend, no la interfaz

8.4. Registro dirigido

`TRACE` está desactivado por defecto porque a 415 datagramas/s inunda cualquier fichero. Para usarlo con criterio:

```
[logging]
level = "info"
overrides = { "pw::state" = "trace", "pw::decoder" = "debug" }
```

---

9. Captura de sesiones UDP

9.1. Configuración del juego

`Game Options → Settings → UDP Telemetry Settings`:

Ajuste	Valor para capturar
UDP Telemetry	On
UDP Broadcast Mode	Off
UDP IP Address	`127.0.0.1` (o la IP del PC si se juega en consola)
UDP Port	`20777`
UDP Send Rate	El máximo disponible
UDP Format	**2026** para el Season Pack, **2025** para las capturas del formato anterior
Your Telemetry	Public para la captura propia; Restricted para el escenario multijugador
Show Online ID	On si se quiere capturar el escenario con nombres

9.2. Capturas necesarias

Prioridad de la fase F1.4:

#	Escenario	Duración	Prioridad
1	Carrera 2026 con 24 coches, con flashback, parada y colisión	20 min	**Crítica**
2	Carrera 2025 equivalente	20 min	**Crítica**
3	Multijugador con rivales de telemetría restringida	10 min	Alta
4	Clasificación → carrera en el mismo fin de semana	15 min	Alta
5	Sesión con Safety Car y bandera roja	15 min	Media
6	Time Trial	5 min	Media
7	Modo espectador	10 min	Baja

9.3. Después de capturar

```
pw-capture verify   captura.f1raw
pw-capture stats    captura.f1raw
pw-capture anonymize captura.f1raw --out captura_anon.f1raw   # OBLIGATORIO antes de versionar
pw-capture slice    captura_anon.f1raw --from 00:05:00 --to 00:06:00 \
                    --out tests/scenarios/2026_flashback_simple.f1raw
pw-capture extract  captura_anon.f1raw --packet 16 --index 0 \
                    --out tests/fixtures/packets/2026/16_car_telemetry2.bin
```

> **La anonimización no es opcional.** Una captura sin anonimizar contiene nombres de otros jugadores dentro de los paquetes 4 y 9 (PRV-03).

---

10. Replay

Necesidad	Orden
Ver una sesión	`pitwall --replay s.f1raw`
Depurar despacio	`--speed 0.25x --log-level trace`
Prueba de carga	`--speed 4x` (4× la carga real)
Prueba de integración	`--speed max --headless`
Probar el receptor de verdad	`pw-capture replay s.f1raw --mode udp --port 20777`
Regenerar el catálogo	`pw-capture reprocess s.f1raw`

---

11. Frontend

11.1. Órdenes

```
cd ui
npm run dev        # servidor de desarrollo
npm run build      # compilación estática, servida por el núcleo
npm run test       # pruebas de componentes
npm run test:e2e   # extremo a extremo contra PitWall.exe en modo replay
npm run typecheck
```

11.2. Tipos compartidos

```
python tools/gen_ts_types.py
```

Genera `ui/src/domain/generated.ts` desde las definiciones del dominio en C++. **Nunca se edita a mano.** La integración continua falla si el fichero generado difiere del comprometido.

11.3. Reglas de rendimiento

1. La telemetría de alta frecuencia **no entra en el estado de React**.
2. Toda gráfica decima conservando mínimo y máximo por columna.
3. El mapa usa dos lienzos: estático y dinámico.
4. Al salir de una pantalla, se desuscriben sus canales.
5. Un único bucle de animación compartido.

Estas cinco reglas son la diferencia entre 60 fps y 12 fps. Se comprueban en la revisión de código.

---

12. Integración

12.1. Flujo de trabajo

```
git checkout -b feature/nombre

# Ciclo: escribir prueba → implementar → verificar
ctest --preset debug --output-on-failure

# Antes de subir
cmake --build --preset debug          # 0 advertencias
ctest --preset asan
python tools/traceability.py
python tools/check_deps.py            # grafo de dependencias
cd ui && npm run typecheck && npm run test && cd ..
```

12.2. Lista de comprobación previa a integrar

- [ ] Compilación sin advertencias en MSVC y Clang
- [ ] Todas las pruebas en verde
- [ ] Desinfectantes sin hallazgos
- [ ] Cobertura no ha bajado
- [ ] Los requisitos nuevos tienen prueba etiquetada
- [ ] El grafo de dependencias sigue siendo válido
- [ ] La documentación afectada está actualizada
- [ ] Si cambia un struct del protocolo, los fixtures se han regenerado y el diff se ha revisado a mano

12.3. Normas de Git

Son permanentes y se aplican a todo el repositorio.

Autoría. Los commits pertenecen **únicamente** a la identidad configurada en Git por el autor del proyecto. Una herramienta de IA puede ayudar, revisar, escribir código y proponer mensajes de commit, pero **nunca** figura como

Author, *Committer* ni coautora. Está prohibida **cualquier** atribución de autoría a una IA, en particular trailers del tipo `Co-Authored-By: <herramienta de IA>` o cualquier otro equivalente, sea cual sea la herramienta.

Commits.

- Pequeños y coherentes: **un commit por bloque lógico** de trabajo, no por fichero ni por día.
- Antes de cada commit: revisar el diff completo, ejecutar las pruebas, actualizar la documentación afectada y comprobar `git status` (sin ficheros inesperados ni restos de pruebas).
- No se hacen commits automáticos: sólo cuando el bloque ha sido revisado y el autor lo autoriza.
- Comprobar la autoría antes de publicar: `git show -s --format=fuller HEAD`.

Publicación.

- No se hace push de trabajo no revisado.
- Si hay que reescribir historia ya publicada, **sólo** con `git push --force-with-lease`. Nunca con `git push --force`: `--force-with-lease` falla si el remoto ha cambiado desde la última lectura, y así no se pisa trabajo ajeno sin verlo.

12.4. Documentación

```
cd tools
node md2pdf.mjs ../docs/01-REQUIREMENTS.md
node build-all-pdfs.mjs        # todos los documentos
```

El `.md` es la fuente. El `.pdf` es un artefacto generado y **nunca se edita**.

---

13. Pruebas manuales

Lo que no se puede automatizar, con lista de comprobación para que al menos sea repetible.

13.1. Sesión de conducción (cada hito)

- [ ] Los valores de la telemetría se corresponden con lo que hace el coche
- [ ] La torre de tiempos coincide con el HUD del juego
- [ ] El mapa muestra los coches donde están
- [ ] Nada distrae ni parpadea durante la conducción
- [ ] Las cifras se leen a 1 m de distancia
- [ ] La latencia se percibe como instantánea

13.3. Sesión con `pw-listen` y el juego real (F1.1-13)

`pw-listen` no decodifica nada: sólo demuestra que los bytes entran bien. Para la referencia de ~415 datagramas/s, **el juego debe estar a 60 Hz**; a 20 Hz la tasa será menor y eso no es un fallo.

- [ ] `cmake --build --preset release --target pw-listen`
- [ ] Arrancar `pw-listen --port 20777` (o `--bind 0.0.0.0 --port 20777` para consola/LAN)
- [ ] En el juego: *Settings* → *Telemetry*, UDP activado, IP del PC, puerto 20777, **60 Hz**
- [ ] Entrar en una sesión: el estado pasa de `escuchando` a `recibiendo`
- [ ] El ritmo ronda los **415 pkt/s** (referencia, no igualdad exacta) y ~0,40 MiB/s

- [ ] Un solo origen, con la `IP:puerto` esperada
- [ ] Tamaños observados coherentes con `03-UDP-PROTOCOL.md` §3 (el mayor, 1470 B < 2048 B de buffer)
- [ ] `pool_agotado=0`, `qraw_drop=0`, `qrec_drop=0`, `truncados=0`, `oversized=0`
- [ ] Salir de la sesión o cerrar el juego: el estado pasa a `inactivo` tras el silencio
- [ ] Volver a emitir: vuelve a `recibiendo`
- [ ] `Ctrl+C`: imprime el resumen, `buffers en uso al terminar: 0` y sale con código 0

13.4. Recepción desde la LAN (DoD de F1.1)

- [ ] `pw-listen --bind 0.0.0.0 --port 20777` en el PC
- [ ] El emisor (consola u otro PC) apunta a la **IP del PC**, no a `127.0.0.1`
- [ ] Llegan datagramas y el origen mostrado es el del emisor
- [ ] Si no llega nada: cortafuegos de Windows, IP equivocada o el juego emitiendo a otro puerto

> Que el `bind` a `0.0.0.0` funcione **no** demuestra que el cortafuegos y la LAN estén bien: eso es justo lo que comprueba esta prueba manual.

13.2. Configuración en consola (al implementar RF-003)

- [ ] La consola encuentra el PC con la IP configurada
- [ ] Los datagramas llegan con bind `0.0.0.0`
- [ ] Los nombres se muestran según la plataforma

13.3. Tras un parche del juego

- [ ] `m_gameMinorVersion` ha cambiado y se registra
- [ ] `bad_size`, `unknown_id` y `unknown_version` siguen a cero
- [ ] Las capturas anteriores siguen reproduciéndose
- [ ] La suite completa sigue en verde

---

14. Resolución de problemas frecuentes

Problema	Causa habitual	Solución
No compila tras clonar	Dependencias sin resolver	`cmake --preset debug` de nuevo; comprobar red
`static_assert` falla en un struct	Transcripción incorrecta	Comparar campo a campo con la especificación. **El número oficial es la verdad**
Cero datagramas con el juego abierto	UDP desactivado, IP o puerto distintos, cortafuegos	Seguir la lista de la pantalla de *Estado*
Datagramas «desconocidos»	Otra herramienta emitiendo al mismo puerto	Cambiar el puerto, o activar el filtro por origen
Todo a cero en los rivales	Telemetría restringida: **es correcto**	Comprobar que se muestra como «Restringido», no como 0
Vueltas duplicadas	Flashback mal gestionado	Reproducir el escenario con `TRACE` en `pw::state`
Frontend a 12 fps	Telemetría dentro del estado de React	Revisar las cinco reglas de §11.3
La ventana abre pero no recibe nada	El núcleo falló al arrancar	La pantalla de Estado lo explica; mirar también el log
Texto borroso al mover entre monitores	Falta la declaración de PPP por monitor v2	Revisar el manifiesto de la aplicación
Fugas en el replay	Buffers no liberados	Ejecutar con el desinfectante de direcciones
Replay más lento que 1×	Resolución del temporizador	Comprobar que se solicita 1 ms

15. Por dónde empezar mañana

Si esta guía se lee al inicio de la primera sesión de desarrollo, el orden es:

1. `11-IMPLEMENTATION-PLAN.md` §3 — **FASE F0**, tareas F0-01 a F0-14.
2. Cerrar la DoD de F0 antes de tocar nada más.
3. Continuar con F1.1 y F1.2a, que pueden solaparse.
4. Priorizar **F1.4 (grabación y replay)** en cuanto haya recepción: es lo que libera del juego para el desarrollo diario.
5. Y **F1.6b (shell de escritorio)** antes de construir pantallas: pone el entorno definitivo y responde las dos preguntas abiertas del WebView2 (Q-10 y Q-11).

Fin de `PW-DEV-016` v1.1